

The 68040 Processor:

Part 2, Memory Design and Chip Verification

The second of a two-part series about the design of the 68040 discusses the memory subsystem, the external bus, chip and board testing, and design-verification methods.

*Robin W. Edenfield
Michael G. Gallup
William B. Ledbetter, Jr.
Ralph C. McGarity
Eric E. Quintana
Russell A. Reininger*

Motorola

The 68040 is a third-generation, full-32-bit microprocessor in the Motorola 68000 family. The 68040 integrates over 1.2 million transistors on one chip and is the first microprocessor to be fabricated with 0.8-micrometer CMOS (complementary metal-oxide semiconductor) design rules. It executes the complete 68020 microprocessor and 68882 floating-point coprocessor instruction sets. The sustained performance level is 20 VAX-equivalent MIPS (million instructions per second) and 3 Mflops (million floating-point operations per second) at a clock speed of 25 MHz. Pipelined integer and floating-point execution units that operate concurrently with separate internal memory controllers and an autonomous bus controller contribute to this performance level. Physical caches of 4 Kbytes each for instruction and data reside on chip. Separate address-translation caches of 64 entries apiece that operate in parallel with the instruction and data caches combine with each physical cache. This arrangement provides complete memory management in a virtual, demand-paged operating system.

In the February 1990 issue of *IEEE Micro*, we described design trade-offs, provided some insights into processor implementation, and detailed the integer and floating-point units. Here we discuss the memory subsystem, the external bus, chip and board testing, and design-verification methods.

Memory subsystem

The 68040 memory subsystem contains the instruction memory unit, the data memory unit, and the bus controller (see Figure 1). The subsystem provides instructions and data to the integer unit (IU) and the floating-point unit (FPU, not shown in figure) at a rate that matches the chip performance re-

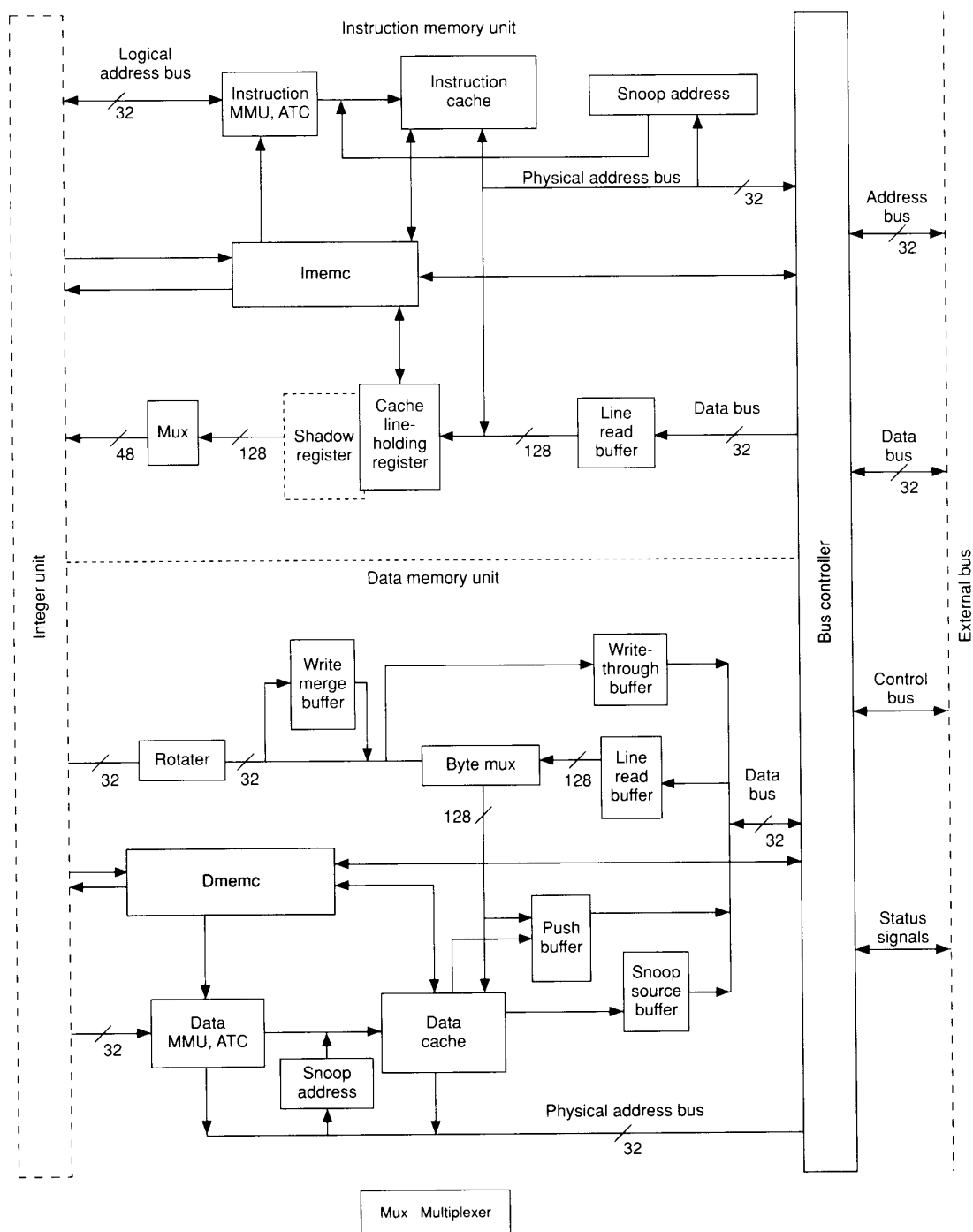


Figure 1. Memory subsystem and data paths.

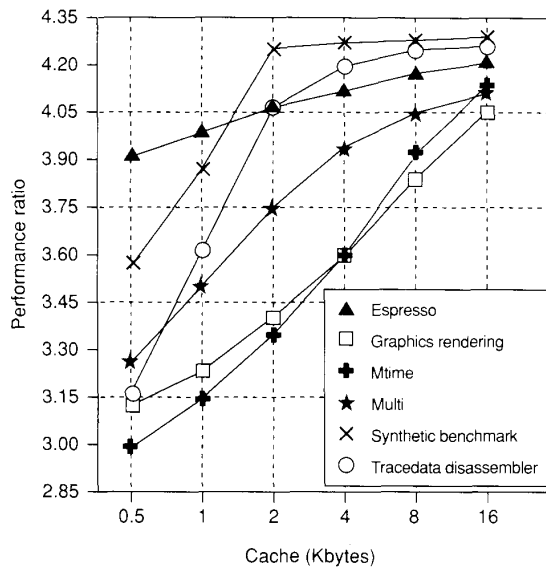


Figure 2. Processor performance relative to the 68020 versus cache size (where the 68020 equals 1).

quirements. Based on the data found in MacGregor, Mothersole, and Moyer¹ and on our analysis of trace data, we estimated that the IU would require transfer speeds of approximately 48 Mbytes per second from the instruction stream and 32 Mbytes/s from the data stream. Because we made the fundamental decision to have one external bus, we optimized the memory subsystem so that the majority of the required bandwidth comes from internal caches. The external bus handles cache load and unload functions. To this end, we chose a Harvard architecture memory subsystem with two memory management units (MMUs), two caches, and three separate controllers that function as follows:

- The data memory controller (Dmemc) maintains the data cache and its MMU.
- The instruction memory controller (Imemc) maintains the instruction cache and its MMU.
- The bus controller manages external bus protocol.

The three independent controllers and their associated buffers decouple external bus effects and cache management (loading or unloading) from instruction execution. Since the three controllers are independent, a request/acknowledge protocol handles communication between the two memory controllers and the IU or the bus controller. The IU requests data or instructions from the two memory controllers, and they provide data to service IU requests. The controllers also make requests of the bus controller that cannot be satisfied by the caches. Similarly, the external bus controller can re-

quest cache activity to satisfy requests from the external bus (snoops).

The autonomous controllers and buffers are essentially a pipelined memory system that isolates the IU from external bus activity, as Figure 1 shows. Figure 1 also illustrates the buffering between the IU and the memory subsystem as well as the buffering between the external bus controller and both internal memory controllers. We discuss this buffering later, in addition to the address-translation caches and cache line holding registers.

Internal caches

We chose the cache size and organization based on simulations performed on trace data with a cache simulator program. This simulator allowed us to run the collected instruction and data sequences with various cache sizes and organizations. Figure 2 shows some of the results of these simulations on various types of traces. These traces demonstrate a wide range of effectiveness provided by different cache sizes. From this data, we chose 4-Kbyte caches as a balance between performance requirements and silicon constraints. The hit rates that the simulator calculated from this data are consistent with those given by Smith.²

The data cache and the instruction cache are both four-way set-associative units with 64 sets of four entries (lines) each. Each cache line in each set contains 16 bytes of data for a total size of 4 Kbytes per cache. We chose the four-way set-associative organization to maximize hit rates in the caches while minimizing total silicon area. We preferred a fully associative cache to maximize hit rates. However, the amount of silicon required for a fully associative cache was prohibitive. We also considered direct-mapped and two-way associative caches, but the four-way set-associative organization provided superior performance without excessive silicon cost.^{3,4} Results from the cache simulations performed on the trace data also supported this decision.

Since the memory subsystem provides logical-to-physical address translation by the MMUs, another fundamental design decision employed physical addressing for both caches. This approach makes the caches into mirrors of external, physical memory. Therefore, the operating system does not need to flush data or instructions from the caches on a process switch. Physical caches allowed us to provide external access to the caches (bus snooping) without requiring a reverse physical-to-logical address translation. Having physical caches also removed the problems associated with virtual address aliasing in the caches. Note that physical addressing into the caches and a 4-Kbyte minimum physical page size allow cache lookup concurrent with logical-to-physical address translation. These concurrent operations can occur due to the fact

that bits 11 through 0 are untranslated for a 4-Kbyte page. Cache lookup concurrent with address translation by dual MMUs guarantees that a cache lookup can complete in one clock cycle.

In addition to data or instructions, each entry in the caches contains an address (tag) and state information. This information consists of a validity bit in the instruction cache and a validity bit and four dirty (incoherent) bits in the data cache. Each dirty bit corresponds to a long word (32 bits) in the cache line. The data cache can handle writes to memory in two modes. In the write-through mode, each write to memory proceeds to external memory. The write also proceeds to the data cache when the address resides in that cache. In this mode, the Dmemc clears the dirty bits to indicate that the cache and external memory have the same data and are therefore coherent. In the copy-back mode, each write to memory proceeds only to the data cache. The appropriate dirty bit or bits are set to indicate that the cache line and external memory are incoherent (the cache line is dirty). The system writes (pushes) a dirty line to external memory when that entry location is allocated for another line of data.

We chose the line-validity method to reduce the number of bits required for state information. Line validity implies that external memory loads an entire line into the cache at once. Reading and loading an entire line benefits the instruction cache. Instruction accesses are largely sequential, and fetching the entire line is effectively an extended prefetch operation. The benefit in the data cache is less obvious because data accesses are not generally sequential. However, trace-data analysis indicated that loading the entire line into the data cache was advantageous. Further, we minimized the penalty for fetching an entire line of data by allowing cache accesses to occur in the processor concurrently with the external bus activity required for cache loading. We also designed the external bus protocol to use line-burst transfers from external memory to internal buffers without using internal cache buses.

Cache-maintenance activity refers to the line reads required to load the cache or to the pushes required to move a dirty line from the cache to external memory. When an access for the IU misses in the data cache—and the line to be replaced has dirty data—the Dmemc must perform two sets of actions. It must push the line currently in the cache and fetch the line containing the data requested by the IU from memory and load it into the cache. The Dmemc isolates the IU from cache maintenance primarily by buffering dirty lines (picked for replacement) in the push buffer. Thus the system can read the line before the push occurs on the external bus. This deferral of pushes minimizes the latency from a cache miss to data that is sent back to the IU, as explained later. The fact that cache accesses are allowed while a push is occurring on the external bus further minimizes the effects of the pushes.

The cache simulations that we performed indicated

that 22.2 percent of all pushed lines had only 1 dirty long word, 3.8 percent had 2 dirty long words, 10.5 percent had 3 dirty long words, and 63.3 percent contained 4 (all) dirty long words. Because the additional status contained in each line indicates dirtiness for each long word, the Dmemc can push a line from the cache without requiring a full line burst on the external bus. When only 1 long word in the line is dirty, the Dmemc requests a long-word access to push the cache line.

Pushing 1 long word instead of an entire line saves a minimum of three external bus clock cycles. This savings increases when wait states occur on the external bus. The deferral of pushes, the overlap with cache accesses, and the variable sizes of pushes allow the processor to take maximum advantage of the copy-back mode while minimizing stalls and external bus bandwidth.

The internal Harvard architecture of the processor allows simultaneous accesses to the instruction and data caches. To save power—and since the size of all accesses into the caches is considerably less than 128 bits—the system does not read all 128 bits on every clock cycle. Instead, it reads one half, or 64 bits, of an accessed cache line. Consequently, the Dmemc requires two clock cycles to load a replaced dirty line into the push buffer. However, the Dmemc hides these two cache accesses under the external bus transfer during the line read.

Figure 3 on the next page demonstrates a hit in the data cache, which takes one clock cycle, followed by a miss in the data cache. When the miss occurs, the Dmemc picks a dirty line for replacement and loads it into the push buffer during the beginning of the external bus transfer. As shown in Figure 3, the Dmemc streams data received from memory immediately to the IU. During the remaining portion of the bus transfer, the IU can access the cache. Once the entire line is received from memory, the Dmemc loads the entire line of data (as well as both the tag and state information) in one cycle. The instruction and data controllers both stream data to the IU and allow concurrent bus and cache activity as well.

Although it is also feasible to load a cache as the data is retrieved from memory, we had two reasons for not choosing this type of partial load. Partial loads would have prohibited concurrent cache and bus activity, stalling the IU unnecessarily. Secondly, clean cache lines waiting for replacement remain in the cache until the bus cycle completes. This allows abnormally terminated bus cycles (through a cache inhibit operation or error from the bus) to avoid unnecessarily invalidating an entry in the cache. Delaying cache loading until the bus cycle completes and allowing IU access to the caches concurrently with bus activity reduces the penalty for loading the cache. At most, the penalty amounts to a denial of access to the cache for the one clock cycle required for the load.

In the 68000 family architecture, instruction length

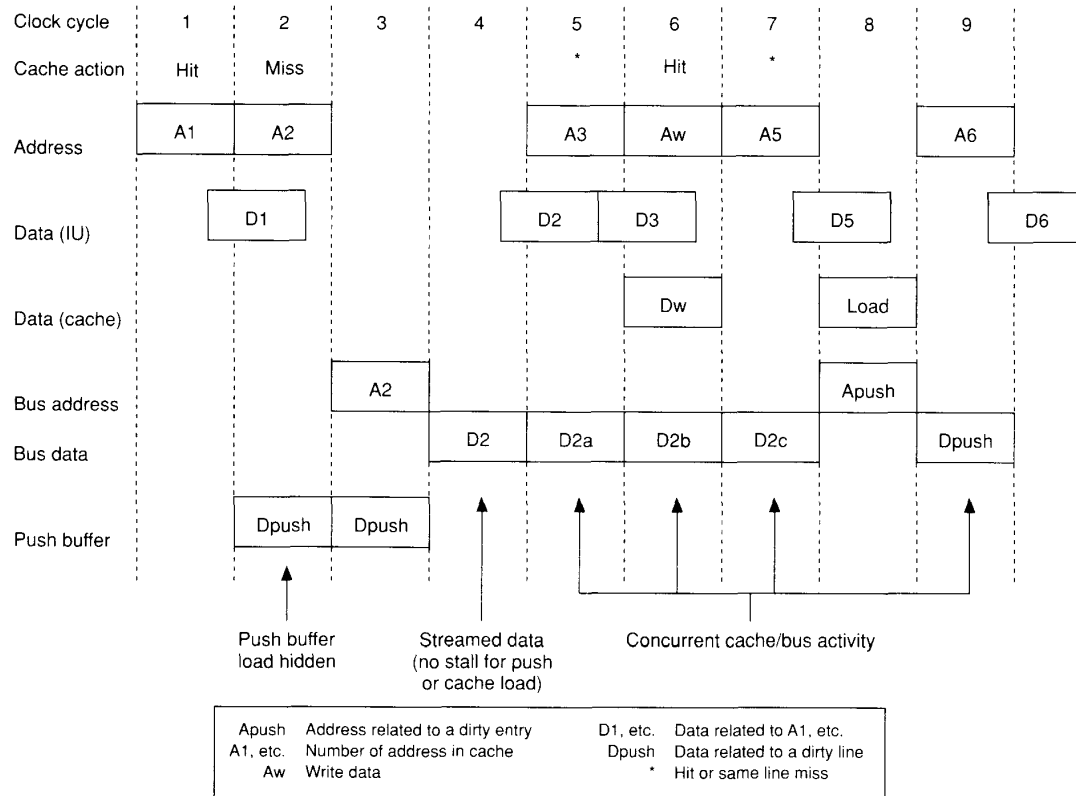


Figure 3. Data memory operation. Each column indicates a data read with overlap.

varies from 1 to 11 words. However, the length for optimized instructions does not exceed 48 bits (3 words). To make sure that the IU receives an optimized instruction every clock cycle, the Imemc maintains a 128-bit cache, line-holding register (CLHR) that is essentially a moving window on the instruction stream. The 48 bits required by the IU are multiplexed out of any word-aligned position in the CLHR. As soon as the last word in either half of the CLHR feeds into the instruction pipeline, the IU requests the Imemc to load that half of the CLHR with a new half line of instructions. This prefetching means that the CLHR does not hold a single cache line—but it can hold pieces of two cache lines.

Since the instruction cache can supply 64 bits in one clock cycle, the Imemc can fill the CLHR in advance of the IU requirement for the next instruction. To improve performance in the case of cache misses, the system loads each long word into the CLHR directly from the external bus as it is received. In addition, the Imemc forwards each long word received from the external bus

to the IU based on the portion of the CLHR the IU requires. This streaming of instructions from the external bus to the IU minimizes the penalty for cache misses.

The system normally fills the CLHR sequentially. However, once a branch has been recognized by the program-counter calculation and decoding stage of the instruction pipeline, the IU requests the Imemc to fetch the target of the branch instruction and place it in the CLHR. The Imemc moves the current CLHR into a shadow register and fetches the target of the branch. If the branch is taken, the system employs the CLHR. Otherwise, it restores the shadow register to the CLHR, thereby avoiding refetching the not-taken path and minimizing branch penalties. When the not-taken path misses in the instruction cache, the Imemc requests the line from the external bus controller. When the instruction pipeline determines that the branch is not taken, further accesses can occur on the branch-taken path concurrently with the line read on the external bus—as long as they all hit in the cache. This “never mind” of

the cache line fetch minimizes the impact of a cache miss for a request down the wrong path of the branch.

On the data side, the variable operand length does not exceed 32 bits. When the IU requests a data operand, the Dmemc analyzes the alignment of the operand and decomposes the request into multiple cache lookups when required.

The fact that the data cache can supply 64 bits of data per clock cycle allows the Dmemc to service any read request that does not cross a half-line boundary in one clock cycle, regardless of alignment. Read hits that cross either a half-line or a line boundary require two clock cycles.

A read that misses requires one external bus transfer when the operand resides within a long word; otherwise, the read requires two external bus transfers. Misalignment within a line on a read miss does not require that the entire line be loaded before the data proceeds to the IU. The Dmemc feeds data to the IU as it is received from the external bus and multiplexes the proper portions of the necessary long words to the IU as well. With no-wait-state operation on the external bus, the IU only stalls two clock cycles on a cache miss for aligned data. The Dmemc informs the IU that its request is satisfied when it receives the last portion of data from the external bus. After the request is satisfied, the Dmemc accepts new requests from the IU. If in fact the next request is for another portion of the line coming from the external bus, the data again proceeds to the IU as soon as it is received from the external bus.

This streaming of data from the external bus directly into the IU helps both IU string operations and FPU double-precision operand loads. Also, if the next request can be satisfied by cache accesses alone, external bus activity does not stall the IU. In fact, the IU remains unaware of it. The Dmemc can service multiple IU requests before the line read completes and the cache line is loaded into the cache.

Once the line read completes, it takes one clock cycle to load the cache line into the cache. Consequently, the IU cannot access the cache for one cycle. This predicament stalls the IU when an operand read access is requested simultaneously with the completion of the line read. Writes do not stall the IU because they are passed off to the Dmemc and stored in the write buffer, thereby allowing the IU to proceed. Since the data cache can write an entire cache line at once, the Dmemc takes only one clock cycle to perform a copy-back write (of any alignment) into the cache, provided that the write operand does not cross a line boundary. When the Dmemc writes to a copy-back page and it misses in the cache, the controller requests a line read from the external bus. The line obtained is loaded into the cache and modified with the data written from the IU. This procedure is called write allocate.

The Dmemc accepts new requests for data accesses from the IU once the first data-transfer acknowledge signal is received from the external bus. Therefore, the

IU does not stall waiting for the cache to be filled for the write miss. For writes to write-through pages, the Dmemc decomposes the request into aligned accesses to the external bus, which can (depending upon alignment) produce multiple accesses in the data cache as a side effect.

Memory management

The 68040's dual MMUs provide logical-to-physical address translation for instructions and data, respectively. Each MMU contains two transparent translation registers (TTRs) and an address translation cache (ATC) of 64 entries organized as four way set associative. Both the TTRs and the ATCs provide 32-bit, logical-to-physical address translation with a 2-bit extension controlled by the user. In addition to address translation, the MMUs provide supervisor and write protection and cache-moding information on a per-page basis. Supervisor-only instructions—including those that query and load an address and flush ATC entries—maintain the TTRs and ATCs. The ATC entries (either supervisor or user) are loaded by a micro-coded table walk algorithm. This microcode uses separate supervisor and user root pointers as entry points to a three-level table structure of single-format, 4-byte descriptors. The ATC provides a 4- or 8-Kbyte selectable page size, and the TTRs provide a transparent mapping of segments from 16 Mbytes to 4 Gbytes in size.

The factors that motivated the design of the MMU were (in roughly this order):

- performance,
- ease of design,
- table memory usage,
- essential requirements,
- chip area, and
- 68030 compatibility.

Table-search algorithms. We analyzed several table-search methods, including hardware two- and three-level table searches and a software table search. Although the software table search method would have been the easiest to implement, we rejected it because it was too slow. Even with significant hardware assistance, a software table search was at least twice as slow as its hardware counterpart, reducing overall performance by 7 percent. Also, the software algorithms that could not use a 68040 hardware assist were 10 times slower than an equivalent hardware version. We felt that users would be unwilling to sacrifice this much performance.

We also evaluated two- and three-level table structures. A two-level structure would reduce the time required to perform a table search but would make operating system operations slower and less flexible

MMU segments

A typical operating system allocates each process its own private address space, which gives each process its own root pointer and table structure. The system allocates virtual memory to a process in units called segments. Segments can vary in size, and they grow or shrink as a process executes. Therefore, the system allocates an entire page table, rather than just a single page, for each segment. Marking a corresponding page descriptor valid or invalid and allocating or deallocating a physical page can add pages to, or delete them from, a segment.

The minimum amount of virtual space a segment claims is thus the amount claimed by a page table. A small segment size has several advantages.

First, the smaller segment requires less physical memory overhead for the associated page table. Less overhead means less waste of physical memory, faster paging of process information to and from the disk, and faster initialization of the tables.

Second, a smaller segment size allows a greater maximum number of segments to be allocated to a process. A trend in operating system design uses "lightweight" processes. One of these processes shares the logical address space of the parent process that spawns it, meaning that it receives one or more segments from the pool available from the parent process. If more segments are available, more lightweight processes can be created. Because these processes typically are small and of short duration, they avoid a great deal of process creation/deletion overhead, which improves performance.

These factors guided us in the selection of the 68040's three-level table structure. Assuming a

4-Kbyte page size, the 68040's structure uses a 7-7-6-12 logical address partitioning: the upper 7 bits of the logical address provide for the initial lookup, the next 7 bits for the second-level lookup, and the next 6 bits for the page-table lookup. The remaining bits are untranslated. Because the descriptors in the tables are all 4 bytes long, ($2 \times 7 \times 4 = 512$) bytes are required for each first- and second-level table, and ($2 \times 6 \times 4 = 256$) bytes are required for each page-level one. We compare this later to a 10-10-12, two-level, logical address partitioning. A 10-10-12 partitioning requires ($2 \times 10 \times 4 = 4$ Kbytes) for each upper level and page-level table.

A typical Unix implementation allocates a minimum of four segments per process (text, stack, data, and miscellaneous). So, the 68040's table structure requires one first-level, one second-level, and four page-level tables. This adds up to $512 + 512 + (256 \times 4) = 2$ Kbytes of overhead for a minimum-size process. A 10-10-12 partitioning requires one upper level and four page-level descriptors, which add up to 4 Kbytes + $(4K \times 4) = 20$ Kbytes of overhead for a minimum-size process. This amount of overhead is an order-of-magnitude more than that required by the 68040.

The 68040 7-7-6-12 partitioning provides a maximum of ($2 \times 14 = 16K$) segments per process, while 10-10-12 partitioning uses a maximum of ($2 \times 10 = 1K$) segments.

Consequently, the 68040's three-level structure results in less page table overhead and permits more lightweight processes to be attached to a parent process.

(see the box). Existing user opinion was split—some requested three levels and others two levels. We could have provided both via a mode bit, but this approach would slow down the table-search algorithm. In fact, we calculated that a three-level modeless table search would be as fast as a two-level moded search (with a three-level search as the second mode). In the face of this information, the performance benefits of a two-level table structure disappeared, so we chose a three-level one.

Page size. A performance requirement of the 68040 specified that the ATC and physical cache lookups must occur simultaneously in the instruction and data sides of the processor. The ATC uses the upper address bits, and the cache uses the lower ones. This requirement constrained the minimum page size because the same bits cannot be used for both lookups. In a four-

way set-associative, 4-Kbyte physical cache, bits 9 through 0 of the logical address are ineligible for use in the ATC lookup. Thus the minimum possible page size was $2 \times 10 = 1$ Kbyte. However, because future derivatives of the part could contain more cache, we chose a more realistic minimum of 4 Kbytes.

A survey of existing computer systems revealed a trend toward larger page size. Most existing systems use a 4-Kbyte page size, while the newer ones employ 8 Kbytes. Designers still debate the ideal page size. A large size increases the efficiency with which data can be paged to and from the disk and reduces the number of ATC faults. However, it also increases internal fragmentation (locations in a page not used by a process), thus increasing the likelihood that unused data is unnecessarily paged to and from the disk. We compromised and provided a mode bit that selects either a 4- or 8-Kbyte page size.

Set associativity versus full associativity. Using the collected trace data, we conducted simulations to determine the best associative organization for the ATCs. We estimated that a fully associative ATC would require twice as much chip area as an equivalent four-way set-associative ATC. Figure 4 compares the two implementations for various ATC sizes and page sizes. The hit rate is essentially the same for 32-entry, fully associative and 64-entry, four-way set-associative ATCs. We chose the latter organization because

- 1) it provided a minor improvement in hit rate;
- 2) the same memory cell could function in both the ATC and in the physical caches, which reduced the design effort; and
- 3) a 64-entry ATC fares better on those rare occasions when users run a program that has a very large working set. (However, the programs on which we collected trace data did not come from this category.)

Note that in Figure 4 the point chosen (at 64 entries) is well past the knee of the curve. This selection provides a margin of safety for instances in which the 68040 operates in an environment that requires significantly more entries than those traced.

As the chip's design progressed, we periodically reviewed the ATC size. We found that the chip size would not be affected if the ATC size was reduced, due to the way other blocks were placed around the ATC.

Compatibility. Performance goals made the 68040 MMUs inherently incompatible with the 68030. Fortunately, all MMU incompatibilities are confined to the operating system, leaving user code completely compatible. Some fundamental incompatibilities exist in the following 68040 characteristics:

- a minimum page size of 4 Kbytes,
- cache-mode selection on a page basis,
- obsolescence of the 8-byte descriptor due to the larger page size, and
- a global bit.

In addition, our desire to improve the performance of table searches (discussed later) caused some incompatibilities. Nevertheless, we attempted to remain as compatible as possible. In fact, when an 68030 operating system uses only 4-byte descriptors and has the same three-level table structure as the 68040, the two are compatible except for the 68040 features just described.

Other issues. Since many operating systems require that supervisor accesses be mapped differently from user accesses, we provided separate user and supervisor root pointers. The appropriate root pointer selection depends on the state of the supervisor bit of the status register. We rejected an alternative method that would

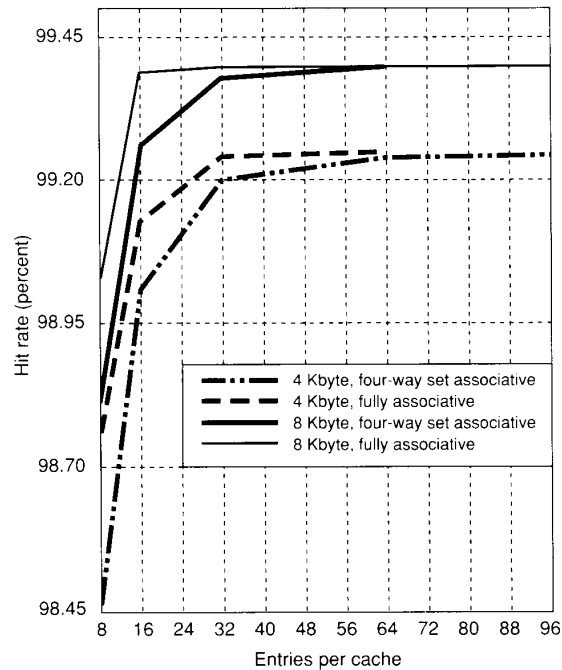


Figure 4. Cumulative ATC.

consider the supervisor bit a thirty-third bit of the logical address and alter the table structure so that the first-level table is 1 Kbyte instead of 512 bytes in size. This approach simplifies the implementation but has the disadvantage of distributing the first-level supervisor table among all processes. It also increases the overhead of changing a supervisor mapping. In addition, it precludes the possibility of paging to disk the first-level table of a process (since the table must be updated when a supervisor mapping changes).

When processes execute, they use common operating system facilities. Supervisor calls access some of these facilities; other facilities simply consist of common subroutines accessed in user space. During execution of a particular process, the ATC contains some mappings that are process specific and other mappings that are common to all processes. The user-specific mappings must be flushed when the execution of one process ends and the next one begins. However, performance improves when the global mappings can remain (since the table searches for these areas do not recur). We added support for this capability by providing a global bit in the page descriptors. This bit selectively flushes entries during ATC flushes. The PFLUSHAN (processor flush all nonglobal) instruction removes only nonglobal entries.

Implementation issues. To reduce the amount of silicon devoted to memory management (based upon the good results in the design of the 68030), we decided table searches should be performed in microcode by the IU. (The IU executes instructions.) This decision did indeed reduce the silicon area required, but it also added numerous and difficult-to-debug boundary conditions. Another complication consisted of the reduction in performance of table searches caused by instruction-stream accesses. These searches must wait for the instruction pipeline to clear (flatten). Flattening the instruction pipeline hurts performance because it creates bubbles or time slots in which no useful activity can take place.

However, we used an implementation technique to increase table-search performance while imposing a negligible increase in silicon area and design complexity. This technique increased the speed of table searches by carefully defining the table descriptors and relying on certain situations only present during table searches.

Normal table-search sequences require three accesses to memory. These accesses cannot occur in a back-to-back fashion because the addresses for the second and third accesses are derived in part from information contained in the descriptor obtained by the prior access. Consequently, we tried to allow a maximum of one dead clock cycle between each access. However, the table-search microcode must accommodate the fact that an upper level descriptor can indicate an invalid mapping. This condition must inhibit further accesses.

An obvious solution to this problem is to slow down table searches. The approach used on the 68030 was to first branch in microcode on the descriptor information and only retrieve the next level when conditions are correct. Instead, the 68040 microcode simultaneously branches on the descriptor information and also requests that the next level be retrieved, relying on the data memory unit to ignore the access if bit 1 of the address is not 0 during a table search.

During a table search, all accesses are 32 bits long and aligned. Thus address bit 1 should normally be 0. The resident bit of a descriptor is only 0 when the descriptor is invalid. So—in the same cycle in which the new address is formed—the resident bit of the descriptor is inverted and gated into bit 1 of the newly formed address. When the microcode branches on bit 1, it relies on the fact that the requested access was ignored.

External bus

This bus provides transfer of cache line data between external memory and the internal caches. We organized these caches to minimize separate bus accesses. We widened the cache line to accommodate more than the natural data requirements of either the instruction pipe-

line or the execution units. We then tuned the external bus and on-chip controller for the efficient transfer of entire cache lines.

The external bus uses a synchronous protocol for the transfer of 8, 16, or 32 bits in a minimum of two clock cycles. The bus transfers 128 bits in a minimum of five clock cycles during a burst transfer. We chose a synchronous bus over the more familiar asynchronous buses found on the 68020 and 68030 designs. We wanted to allow the external bus to scale to higher speeds without making the timing specifications impossible to design within real systems. The synchronous design also allowed for the highest performance interface between the internal bus requesters and external memory in that a cache miss incurs no synchronization delays.

The design supports I/O devices that can transfer bytes, words, and long words on the external bus. In departure from the 68020 and 68030, the external bus does not support dynamic bus sizing. Certainly the vast majority of transfers in high-performance systems based on these earlier microprocessors do not use dynamic bus sizing. We felt that the burden in silicon, external bus performance, and design effort far outweighed the convenience provided to the external system design. In addition, because we considered I/O devices to be part of the operating system environment, the lack of complete compatibility at the bus transfer level was not as important as the performance and design schedule goals.

Protocol. Data transfers on the external bus in the following sequence:

- 1) The processor drives address bits and associated attributes like transfer size and read/writes. It also asserts a transfer-start (TS) signal to indicate that the address bus is valid. In a departure from previous 68000 family members, the 68040 TS does not indicate bus validity at the assertion point, but rather that the bus is valid in a particular clock cycle.

- 2) Within the following clock cycle (at the soonest time), the selected slave responds by asserting a transfer-acknowledge (TA) signal and either drives read data onto the data bus or accepts the write data presented by the processor. A slave can control the transfer rate. If TA is not asserted, the processor simply waits.

- 3) On byte, word, or long-word transfers, the processor accepts only one TA per transfer. Cache line transfers require four TA assertions on valid clock edges (this is the definition of a line burst). On writes, the processor drives new data onto the 32-bit data bus during the clock cycle after each assertion of a TA. On reads, the external memory must place new data on the bus during the clock cycle after it asserts TA.

When a bus error occurs, the responding slave asserts the transfer-error-acknowledge (TEA) signal instead of

a TA. The bus controller design allows a TA or TEA to be asserted on the bus as late as the arrival of data, thereby increasing the effective address-to-response time considerably over previous-generation 68000 family designs.

A bus slave can retry transfers by asserting both a TA and TEA at the same time to the processor. To simplify internal boundary conditions associated with retry and bus-relinquish situations with snooping, the protocol only allows a retry on the first long-word transfer in a burst.

The protocol accommodates nonbursting, 32-bit-wide memory as a sort of "macro," bus-sizing, transfer mode. At the termination of the first long word on the bus, the bus slave responds either by accepting write data or by providing the read data. Along with the TA, the slave asserts a transfer-burst-inhibit pin to indicate that the processor must enter the "fake" burst mode to transfer the remaining 3 long words. The bus controller then issues three separate long-word transfers, rolling the addresses appropriately. This option is useful for external ROM since accesses to ROM are not usually required to operate at full bus performance. Because an entire line is transferred, the data enters into the appropriate cache.

Arbitration. Current members of the 68000 family use an on-chip arbiter for the external bus.⁵ This arbiter allows an external bus master to request the bus from the processor at any time but only grants the bus ownership when a read-modify-write (RMW) sequence is not running on the bus. RMW sequences occur when instructions need to access more than one operand and prevent other bus masters from interrupting those accesses. The other 68000 family members prevent another bus master from using the bus while an RMW is in progress.

As we talked to users, we found that some system designs need to allow alternate bus masters to use the bus in the middle of one of the "indivisible" RMW sequences. A general problem occurs when a system has both on-board RAM and a processor, as well as an off-board bus to other system memory. Other processors or DMA devices on the backplane bus can access on-board RAM as part of the address space of the overall system memory. A deadlock can occur when the on-board processor begins an RMW transfer sequence to on-board memory just as an off-board processor or a DMA device accesses the same memory. Although the system designer can work around the deadlock problem, the known procedures increase the system cost considerably.

The external bus-arbitration scheme allows more flexibility in the design of board-level arbitration. We decided to move the arbitration unit off chip to allow the system designer to choose the arbitration scheme. Simply removing the bus-granting signal to the processor can interrupt locked sequences at any point. The

procedure requires alternate bus-master transfers that do not involve an access to data that is the object of a locked sequence. A system designer can allow those transfers to be the highest priority on the bus to obtain the absolute minimum bus latency. Consequently, the external bus arbiter must be aware of locked sequences. The arbiter must also prevent a locked sequence from being interrupted when

- the interruption is to service another bus master, and
- the bus master accesses the same data that is the object of the locked sequence.

Snooping. The external bus supports a uniprocessor snoop protocol to both on-chip caches. The protocol allows high-performance, off-chip DMA to efficiently use the bus in uniprocessor environments with the data cache in copy-back mode. The protocol does not directly support multiprocessor environments because we chose to maximize uniprocessor performance. This protocol supports both reading from and writing to on-chip cache data and is controllable on a per-transfer basis. The data cache can be updated on snoop writes and supplies data to the snoop master on reads (when a line is dirty). The protocol also allows the instruction cache to be invalidated on writes.

Each bus transfer by an alternate master indicates on the bus (via the pins) what kind of snooping should be performed by the processor: inhibited, read source, read source with invalidate, or write to data cache with invalidate in instruction cache. In general, each bus access that is marked "snoopable" costs one or two cache lookups into the data cache (depending on the bus transfer size) and zero or one lookups into the instruction cache. A snoop lookup in either cache may cause a stall in the IU or the FPU, depending on internal activity. Note that on noninvalidating reads the instruction cache is not accessed because it does not have dirty data.

One can think of the read source with invalidate snoop type as a distributed cache push operation in which the push happens on demand from the external bus instead of from internal sources. Users indicated that the effective use of large, on-chip caches with DMA operations requires something faster than the serial execution of cache management instructions in terms of overall system throughput. This snoop type gives system designers a fast way to move data from memory (either in the on-chip caches or in external memory) to the disk (page-out sequence). When a DMA controller performs line reads to move the data to a disk controller (page-out), it marks the snoop type as read source with invalidate. The processor then accesses both caches. The access to the instruction cache invalidates any line within the cache. The access to the data cache requires it to load the snoop source buffer and then invalidate any line within itself. The bus

controller parcels this line to the DMA controller from the snoop source buffer, effectively moving the line to memory. This invalidation procedure means that the DMA controller can mark bus transfers nonsnoopable during block moves or page-ins from the disk to memory because the lines are not in either cache. This minimizes the effect that snooping has during a page-in, page-out sequence on instruction execution.

We decided not to provide duplicate cache tags because we considered snoop performance less important than raw instruction execution. Devoting silicon area to a duplicate tag structure for snoops would reduce the area available for on-chip cache by approximately 30 percent.

Timing specifications. The processor has two separate input clocks: the bus (Bclk) and the processor (Pclk). All external bus timing refers to Bclk, while internal timing derives from Pclk. (However, all internal operations cycle at the Bclk rate.) Bclk must be exactly half the frequency of Pclk. Internally, Pclk provides timing reference points within each Bclk period. A phase-locked loop provides a close internal relationship between the internally derived timing based on Pclk and Bclk. The external bus uses Bclk to provide synchronous timing for bus transfers.

Early in the design process, we looked at the advantages of using asynchronous clocks. We rejected this approach due to the internal synchronization problems between the data sources, destinations, and external timing. One of the initial design goals was to provide a bus design and protocol that would minimize the latency from a cache miss to available data in the IU. We saw synchronous timing between the bus controller and the IU as the best way to accomplish this goal.

The compatibility of the external bus to existing TTL (transistor-transistor logic) appeared to be an important requirement for this generation of microprocessor. We anticipate advances in TTL technology that will allow zero-wait-state buses to be built for high-performance microprocessors. Users indicated that system cost would require most logic to remain TTL-compatible in the near future.

In prior Motorola microprocessors,⁵ all timing specifications assume that output buffers are driving a lumped capacitive load. With the external bus clock speed increasing to 25 MHz and beyond, many users have had difficulties designing systems using the lumped-load model because it is not an accurate representation for real designs. This problem occurs primarily because the output buffer source impedance must decrease to drive a capacitive load faster. With a lower output impedance, the transmission-line effects of a fast output buffer become more evident in such system problems as line ringing and induced noise.⁶

Data, address, and control lines in the 68040 are individually selectable by group. They can drive either an unterminated, controlled-impedance transmission

line or a terminated transmission line in the highest performance systems. The unterminated buffer drives a 30-ohm loaded bus with minimal ringing (source terminated). The terminated buffer has a low output impedance designed to drive a 50-ohm, DC or AC, terminated transmission line. With this environment, the fast rise and fall times of the output buffers result in high-performance bus designs without the problems associated with impedance mismatches.

The MOVE16 instruction

Since the 68040 provides fast on-chip, floating-point operations and integral memory management, we did not include the 68020 coprocessor protocol in the design. However, the processor still must move data and instructions between main memory and external system hardware such as a graphics engine. Consequently, we decided to include a method to provide higher performance than was available with the coprocessor interface on previous designs.

We had already added the hardware on chip to buffer cache line refills and pushes to minimize the effects of cache misses and the write-back of dirty cache data. We next examined the capability of this hardware to support a memory controller instruction in the instruction set to move a cache line between two memory addresses. This investigation resulted in the MOVE16 instruction. This simple instruction moves a cache line from one address—which may reside in the data cache—to another address outside the cache. The instruction is not a true data type since the IU cannot operate on the data, but MOVE16 does allow the processor to move data quickly on the external bus. It simply loads the line buffer in the on-chip data memory unit with the read data and writes it to a different address. The burst transfer type on the bus sustains the highest performance transfer of a cache line between two memory addresses that this bus can provide.

We compared this instruction with existing instructions. Our performance model predicted that an unrolled loop of four MOVE.L instructions (memory to memory) would sustain one half the transfer rate of a simple MOVE16 loop. The actual chip demonstrated this performance improvement. The execution pipeline can hide most of the loop overhead on top of the external bus transfers to provide this performance. The MOVE16 instruction can provide programmers with nearly all of the available external bus bandwidth, allowing low-cost system designs to perform DMA-like operations without the cost of a separate DMA controller.

Chip and board test

We designed the processor with testability in mind

for both the user and the factory. For the user, the processor includes dedicated test logic that is fully compliant with the *IEEE 1149.1 Standard for Test Access Port and Boundary-Scan Architecture*.⁷ Problems with testing high-density circuit boards led to the development of this standard under the sponsorship of the IEEE Test Technology Committee and the Joint Test Action Group (JTAG). The 68040 supports circuit-board-level test strategies based on this standard. For factory testing, the processor employs structured design techniques for random logic and special test modes for embedded arrays. We tested the execution data paths by using functional test vectors. We believe that this mix of design-for-test techniques provides a good trade-off for circuit area overhead and chip testability. The user and factory testing logic are completely independent. No interfaces to the factory testing logic exist through the IEEE 1149.1-based port.

IEEE 1149.1. The test access port (TAP) and boundary-scan interface include the four required test pins TMS, TDI, TDO, and TCK, and the reset pin TRST. The boundary-scan register consists of a shift register that provides access to all chip clock and signal pins along with associated control signals. The implementation includes the three mandatory instructions BYPASS, SAMPLE/PRELOAD, and EXTEST, but does not include the optional public instructions. The latter instructions provide additional test capabilities.

Five unique instructions can be loaded into the 3-bit instruction register: BYPASS, EXTEST, SAMPLE/PRELOAD, BYPASS2, AND EXTESTZ. The BYPASS instruction selects a bypass register and forms a single-bit shift-register path from the serial test data input pin TDI to the output pin TDO. This configuration allows shifting through the bypass register when testing other devices that share the common serial input line. The EXTEST instruction selects the boundary-scan register. By using the TAP controller, the register can scan user-defined values into the output buffers and capture values presented to input pins. It can also control the direction of bidirectional pins and enable the output drive of three-statable output pins. The SAMPLE/PRELOAD instruction provides a means to capture a snapshot of system I/O signals and to initialize the boundary-scan register prior to the selection of the EXTEST instruction. The EXTESTZ instruction disables the drive on output-only pins, and the BYPASS2 instruction provides a bypass mode without clocking restrictions.

Factory testing functions. As mentioned, the processor uses three different design-for-test techniques for addressing the issues of factory testing. We analyzed trade-offs for each area of the design to see which test technique would best fit each section.

We picked a functional style for the data-path elements for historical reasons. A great deal of functional

tests existed from previous designs. For the random logic and ROM/PLA (programmable logic array) areas, we employed a structural style of design for test. We used existing latch structures to form scan chains, which allows automatic generation of test patterns for these structural areas. For the remaining parts of the processor—specifically the cache and cache tags—special test modes came into play. These test modes allow the caches and tags to be accessed like RAM structures from the pins of the processor. In this way, pipelined one-cycle accesses are available to the caches and tags. This capability allows the use of any type of memory test. In the design of these test modes, we determined that no one or two types of memory test would meet all the needs of factory test. Consequently, we developed a general interface to the internal memory circuits.

Design and verification

We designed the processor in a top-down fashion. From the user specification, we divided the chip into three blocks: the IU, the FPU, and the memory subsystem. We subdivided the three sections into control stores, random logic, and data-path elements for the IU and FPU, data and instruction cache/tag memory, ATCs, internal memory controllers, and the external bus controller. Our modular approach divided the design into pieces that exhibited minimum interaction with each other.

Once the components of the processor were decided and the interfaces were worked out, the process of high-level modeling began. We chose Verilog (verilog_ref)⁸ for the high-level models. These models provided design specifications for each section of the chip, which totaled approximately 36 separate sections. The style of the modules mixed register-transfer-language and functional description types of approaches. The RTL style dominated the data-path sections, while a functional style prevailed in the controller sections.

After each of the behavioral models was debugged, we performed the process of implementation. For the data-path sections, a simple transformation of the RTL style model served as a register data path. The more random areas required a more automated logic synthesis approach. We relied on an internal tool to extract verbatim the logic specified in the behavioral models for the Dmemc and the bus controller. This extracted logic was then fed through Espresso⁹ and MisII¹⁰ from the Berkeley Oct tool set to produce layout-ready transistors. These devices were then synthesized directly into layout.

We relied on a hierarchical method of design verification. This choice came from the observations that each section needed different verification requirements and that the sections needed to be debugged independently. Also, we wanted to evaluate design changes and design error fixes as quickly as possible. Two methods

provided early verification of the modules: instruction sequences for the IU and FPU and dedicated stimuli for other modules.

The dedicated stimulus for the memory subsystem emulated the boundary interfaces so that each section could be debugged separately. This low-level verification then extended to levels in which several memory elements were placed together. We used this method for all three memory subsystem controllers.

We used instruction sequences for stimuli in the IU and FPU stand-alone sections. Early simulation—as in the memory section—allowed us to evaluate design changes. At the last stage of the verification process, we simulated the entire design with these same instruction and data sequences.

This hierarchical verification environment also provided a framework for the verification of the structural, or gate-level, description. Simply replacing a behavior section with a gate representation allows both descriptions of the design to be verified with the same stimulus. Thus we could verify each of the gate-level representations at larger and larger configurations.

We discussed the memory subsystem, the external bus, chip and board testing, and design-verification methods in the 68040. We concentrated on the trade-offs we made to give the 68040 greater parallelism, fewer dead cycles, and consequently better performance than any other comparable microprocessor. ■

Acknowledgments

We thank all of the people who have worked on the 68040 design and have helped to make its implementation possible.

References

1. D. MacGregor, D. Mothersole, and B. Moyer, "The Motorola MC68020," *IEEE Micro*, Vol. 4, No. 4, Aug. 1984, pp. 101-118.
2. A.J. Smith, "Cache Evaluation and the Impact of Workload Choice," *Proc. 12th Int'l Symp. Computer Architecture*, IEEE Computer Society Press (microfiche), Los Alamitos, Calif., 1985, pp. 64-75.
3. C. Alexander et al., "Cache Memory Performance in a Unix Environment," *Computer Architecture News*, Vol. 14, No. 3, June 1986, pp. 41-70.
4. J.E. Smith and J.R. Goodman, "Instruction Cache Replacement Policies and Organizations," *IEEE Trans. Computers*, Vol. C-34, No. 3, Mar. 1985, pp. 234-241.
5. Motorola, *MC68020 32-Bit Microprocessor User's Manual*, Prentice-Hall, Englewood Cliffs, N.J.
6. W.R. Blood, Jr., *MECL System Design Handbook*, Motorola, Phoenix, Ariz., 1988.
7. *IEEE Standard 1149.1, Standard for Test Access Port and Boundary-Scan Architecture*, IEEE Press, Piscataway, N.J., 1990.
8. P. Johnson, "Mixed-Level Design Tools Enhance Top-Down Design," *Computer Design*, Vol. 29, No. 1, Jan. 1989, pp. 85-88.
9. R.K. Brayton et al., *Logic Minimization Algorithms for VLSI Synthesis*, Kluwer Academic Publishers, Hingham, Mass., 1984.
10. R.K. Brayton et al., "MIS: A Multiple-Level Logic Optimization System," *IEEE Trans. Computer-Aided Design*, Vol. CAD-6, No. 6, Nov. 1987, pp. 1062-1081.



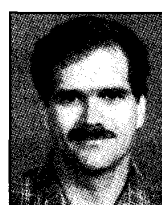
Edenfield



Gallup



Ledbetter



McGarity



Quintana



Reininger

Robin W. Edenfield is a principal staff engineer at Motorola and was primarily responsible for system design of the cache and memory management controllers for the 68040. He previously worked in the design, test, and production of microprocessor systems at several other companies. His current interests include microprocessor architecture and memory subsystems.

Edenfield received the BSEE degree from the Georgia Institute of Technology, where he was elected to Eta Kappa Nu.

Michael G. Gallup, with the microprocessor group at Motorola since 1981, is a circuit design engineer and a member of the technical staff for the high-end design group. His areas of interest include semiconductor devices, digital systems, and computer programming.

Gallup received the BSEE and MSEE degrees from the University of Nebraska at Lincoln. He is a member of the IEEE Computer and Circuits and Systems Societies.

William B. Ledbetter, Jr., is a principal staff engineer in high-end design at Motorola. He was responsible for the external bus specifications and protocol—as well as the system design of the bus controller—for the 68040. He also was a member of the architecture design team for internal memory. He previously designed graphics systems for Data General.

Ledbetter received the BSEE and MSEE degrees from Texas A&M University. He is a member of the IEEE Computer Society.

Ralph C. McGarity is a principal staff engineer and was the systems technical project leader for the 68040 project. He has spent seven years at Motorola designing 68000 family parts. His interests include microprocessor architecture design, implementation, and verification.

McGarity received the BSEE degree from Rice University and the MSEE degree from Carnegie Mellon University. He is a member of the IEEE, ACM SIGARCH, Phi Beta Kappa, and Tau Beta Pi.

Eric E. Quintana has been with the microprocessor group at Motorola since 1984 as a staff engineer and is the architect for the floating-point section of the 68040. He previously worked as a design engineer on the 68020 microprocessor and later on the 68882 floating-point coprocessor.

Quintana received the BSEE degree from Texas A&M University.

Russell A. Reininger is a principal staff engineer for high-end microprocessors at Motorola. He was responsible for the system design of the integer unit in the 68040. Before joining Motorola, he worked for six years at Tracor Aerospace in Austin, Texas, where he designed radiation-hardened military computers.

Russell received the BSEE degree from the University of Texas at Austin.

Readers can direct questions regarding this article to Ralph McGarity, Motorola Inc., M/S OE37, 6501 William Cannon Drive West, Austin, TX 78735-8598.

Reader Interest Survey

Indicate your interest in this article by circling the appropriate number on the Reader Service Card.

Low 156

Medium 157

High 158
